Relatório: Refatoração de Testes e Detecção de Test Smells

Disciplina: Teste de Software
Aluno: Gabriel Chagas Lage
Matrícula: 803292

1. Análise de Test Smells

Lógica condicional no teste

O teste de desativação percorre usuários com `for` e decide o que validar por meio de `if`. Isso mistura dois cenários de negócio em um único teste, dificulta a leitura e pode esconder qual comportamento falhou. A suíte limpa separa os casos de usuário comum e administrador em testes independentes.

Captura manual de exceção

O teste de usuário menor de idade usa `try/catch` sem uma asserção que confirme que a exceção foi lançada. Se a implementação deixar de lançar o erro, o teste termina sem nenhuma asserção e passa silenciosamente. A versão limpa usa `expect(() => ...).toThrow(...)`, garantindo a verificação do comportamento.

Teste desabilitado e sem asserções

O teste marcado com `test.skip` não é executado e também não contém asserções. Assim, a suíte não verifica o relatório vazio. A versão limpa implementa esse cenário e verifica a mensagem `Nenhum usuário cadastrado.`.

Teste frágil de formatação

O teste original exige uma linha inteira do relatório, incluindo o identificador gerado aleatoriamente e a formatação exata. Mudanças de apresentação podem quebrar o teste mesmo mantendo o comportamento relevante. A versão limpa valida que o relatório contém o cabeçalho, os nomes e os status esperados.

2. Exemplo de Refatoração

Antes

```
try {
  userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe('O usuário deve ser maior de idade.');
}
```

Esse código não falha quando nenhuma exceção é lançada. Além disso, a asserção fica dentro de um fluxo condicional, o que foi detectado pelo ESLint.

Depois

```
const criarUsuario = () => userService.createUser(
  'Menor',
```

```
    'menor@email.com',
    17
  );

  expect(criarUsuario).toThrow('O usuário deve ser maior de idade.');
```

A refatoração separa a preparação da ação e da verificação, seguindo Arrange, Act, Assert. O matcher `toThrow` também garante que o teste falhe quando a regra de maioridade for removida ou alterada incorretamente.

3. Relatório do ESLint

Na primeira execução, o ESLint encontrou no arquivo original:

- 4 erros `jest/no-conditional-expect` nas asserções dentro do `if` e do `catch`;
- 1 aviso `jest/no-disabled-tests` para o teste marcado com `skip`;
- 1 aviso `jest/expect-expect` porque o teste desabilitado não tinha asserções.

Depois da refatoração, `npx eslint test/userService.clean.test.js` terminou sem erros ou avisos.

4. Validação Final

Os resultados obtidos foram:

- ESLint na suíte limpa: nenhum erro ou aviso;
- Suíte limpa: 7 testes aprovados;
- Suíte completa: 11 testes aprovados e 1 teste original ignorado, preservado para manter o arquivo de comparação sem alterações.

5. Conclusão

Testes limpos tornam cada regra de negócio explícita, reduzem a fragilidade e facilitam a identificação da causa de uma falha. A análise estática complementa a revisão manual ao detectar padrões problemáticos de forma automática e repetível. Em conjunto, refatoração e linting aumentam a confiabilidade e a manutenibilidade da suíte ao longo da evolução do sistema.